

VCF Core: A Semantic Foundation for RDF Representations of VCF

Elias Crum^{1,2,*}, Bart Buelens², Gökhan Ertaylan² and Ruben Taelman¹

¹IDLab, Department of Electronics and Information Systems, Ghent University – imec, Belgium

²Flemish institute for Technological Research (VITO) Mol, Belgium

Abstract

Connecting genomic observations to biomedical knowledge depends on preserving the context that makes each observation interpretable. In the Variant Call Format (VCF), overlooking version rules, field declarations, or sample-specific allele relationships can silently associate evidence with the wrong variant. Existing semantic models leave parts of this context to application-specific conventions, limiting what users can retrieve and integrate without the loss of context. We developed VCF Core, a vocabulary that makes this context explicit and provides version-specific representations for VCF 4.1–4.5, allowing data to be interpreted according to their declared specification. This paper evaluates VCF Core’s specification conformance, robustness, and practical utility through four research questions. Our reproducible workflow combines competency questions, specification-derived tests, negative controls, cross-implementation comparisons, and semantic round-tripping, with explicit expected answers and transparent reporting of passing, failing, and untested requirements. The specification-derived assessment identifies and demonstrates both preservation and direct structural retrieval in all 210 expanded-profile cases, exercising 53 of 94 registered specification requirements. Cross-implementation testing exposed a silent allele-association error missed by a pre-existing VCF-RDF converter’s tests and guided its correction. A synthetic integration experiment further demonstrates that standard SPARQL queries can correctly join local-allele evidence to external annotations while retaining source provenance, without application-specific decoding. These findings show how explicit VCF semantics make operations otherwise dependent on format-specific code available through native graph relationships, providing a tested foundation for reusable conversion and querying. The accompanying workflow can be adapted to other specification-based vocabularies, while independent testing and application to biomedical datasets can extend the evidence for VCF Core and guide its adoption.

Keywords

Variant Call Format, RDF, Genomic data, Vocabulary, Data integration, SHACL

1. Introduction

Healthcare research and personalized medicine depend on integrating clinical observations, molecular data, and biomedical knowledge across institutions and databases. Differences in data structures, identifiers, and terminology complicate their combined use [1, 2]. Semantic Web approaches address this heterogeneity through shared identifiers, explicit relationships, and reusable vocabularies [3, 4], but meaningful integration also depends on preserving the context in which observations were recorded.

The Variant Call Format (VCF) is widely used to exchange genomic variant data [5]. Each record describes a genomic location, reference and alternative alleles, and optional annotations. Sample columns carry observations such as genotype and sequencing depth. Header declarations define fields and how their values should be interpreted [6]. The GA4GH-maintained specifications define this conserved structure across VCF 4.1–4.5, while differing in declarations and constraints [7, 8]. VCF content fields can contain dependencies to other fields, essentially encoding meaning through the structure of the file. For example, genotype 0/2 identifies the reference allele and the second listed alternative allele, which is indexed based on the order in which these alternate alleles appear in the file,

SWAT4HCLS 2027, February 1–4, 2027, Basel, Switzerland

*Corresponding author.

✉ elias.crum@ugent.be (E. Crum); bart.buelens@vito.be (B. Buelens); gokhan.ertaylan@vito.be (G. Ertaylan); ruben.taelman@ugent.be (R. Taelman)

ORCID 0009-0005-3991-754X (E. Crum); 0000-0001-7734-3747 (B. Buelens); 0000-0001-5602-6435 (G. Ertaylan); 0000-0001-5118-256X (R. Taelman)



© 2027 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

while separately an annotation’s cardinality may depend on its header declaration. When observations are integrated with external information, those relationships must remain available.

RDF and Linked Data offer a basis for exposing this context as explicit, addressable relationships alongside links to biomedical resources [9, 10, 11]. This supports the interoperability and reuse sought by the FAIR principles [12], although wider FAIRness also depends on identifiers, metadata, access, and provenance [13]. Our aim is to provide a reusable representation of VCF observations that retains their source context to better enable these connections.

1.1. Related Work

Related work includes reusable semantic models and VCF–RDF conversion or query tools, with some contributions combining both roles (Table 1). The Genomic Feature and Variation Ontology (GFVO), Genome Variation Ontology (GVO), and HERO-Genomics model genomic or clinical concepts. Tools such as VCF2RDF, TogoVar, BioInterchange, AgroLD, and bio-vcf [14] expose VCF-derived data for integration. The GA4GH Variation Representation Specification (VRS) complements these approaches with computable variation descriptions and federated identification, using JSON Schema rather than RDF or OWL; it does not preserve source-file structure [15].

These approaches distinguish source observations from biological interpretations. VCF Core provides a shared, version-aware representation of source observations with links to existing genomic models, allowing converters and queries to reuse VCF interpretation rules.

Table 1

VCF-related approaches and their roles: reusable models, RDF conversion/query tools, or combinations of both.

Approach	Role and focus
BioHackathon <i>vcf2rdf</i> [16]	Converter. Early VCF–RDF conversion, using FALDO for positions and tool-specific terms for other VCF concepts.
GFVO [17]	OWL ontology. Harmonizes genomic features and variation across VCF, GVF, GFF3, and GTF.
BioInterchange [17]	Converter. Emits GFVO from GFF3, GVF, and VCF; its VCF reader extends the GFF3 reader, so records become generic features, and it reads VCF 4.1–4.2 only.
VCF2RDF [18]	Mapping/converter. Isomorphic VCF–RDF mapping exposing file components independently with source context; not a genomic ontology.
TogoVar <i>vcf2rdf</i> [19]	Converter. Simple RDF representation of population allele-frequency VCF data for integration with TogoVar’s biomedical knowledge graphs.
GVO [20]	OWL ontology. Describes biological variation, particularly complex structural variants, rather than VCF structure and interpretation rules.
VariantKG [21]	Converter + application ontology. Builds VCF-derived graphs for variant analysis and graph machine learning, not specification modelling.
SPARQLing genomics [22]	Converter + query platform. Generates RDF from VCF and other genomic sources and serves it for querying; its terms are tool-specific, not a VCF model.
Semantic Beacons [23]	Integration mapping. Maps GA4GH Beacon responses to RDF with FALDO, SO and GENO for federated genomic querying; not a VCF–RDF representation.
HERO-Genomics [24]	Ontology. Integrates VCF- and Beacon-derived information across clinical and research settings, with broader scope than VCF specification coverage.
AgroLD VCF2RDF [25]	Converter. Incorporates VCF variation into AgroLD’s plant-science knowledge graph; not a dedicated VCF ontology.
VCF-RDFizer [26]	Converter + precursor vocabulary. Our earlier VCF–RDF workflow; VCF Core separates model from conversion and narrows scope to the specification.

The VCF Core Vocabulary. Here, we present and assess the VCF Core Vocabulary¹ (*vcfcv*) [27]. It combines source-context preservation, *expanded* and *condensed* sample column representations,

¹<https://ecrum19.github.io/vcf-core-vocabulary/>

123 curated alignments to existing semantic models, and version-specific artifacts for VCF 4.1–4.5. Four questions structure its assessment: **RQ1**: does VCF Core preserve the intended meaning of the VCF constructs it claims to support? **RQ2**: do the version-specific artifacts, the two *sample profiles*, and a semantic round-trip behave as documented in graphs from a separately implemented VCF-RDF converter? **RQ3**: does the resulting representation support a reproducible integration task while retaining source context? **RQ4**: how does VCF Core relate to other existing VCF semantic models?

We assess representation, implementation, and answer correctness separately, combining specification-derived tests, comparison across producers, an integration example, and comparison with related models. The resulting evidence identifies both reusable capabilities and gaps requiring further testing. Throughout, `vcfc:` denotes <https://w3id.org/vcf-core/vocab#>.

2. Methods

2.1. Representation and assessment design

VCF Core was developed iteratively from the earlier VCF-RDFizer vocabulary [26], using AI-assisted drafting and expert human review to derive terms, field definitions, and constraints from VCF 4.1–4.5 [8, 6]. It represents files, header declarations, records, and sample observations as distinct resources, linking each field value to the declaration needed to interpret it (Figure 1). A `VariantCall` groups a record’s observations, while a `VCFSample` identifies a source column; field definitions remain separate from their occurrence in a header. PROV and DCAT support source descriptions, while SO and FALDO connect to alteration and location models without asserting normalized variant identity across files [28, 29, 30]. VCF Core is a semantic vocabulary with classes, properties, declared domains and ranges, and a shallow subclass hierarchy; it carries no OWL restrictions, cardinality, disjointness, or inverse axioms. Value and cardinality constraints are expressed as SHACL shapes [31]. Terms and relationships can be browsed in the *VCF Core Vocabulary Explorer*² [32].

Two *sample profiles* provide alternative representations. The *expanded* profile gives each sample’s field value its own resource; the *condensed* profile stores sample-ordered vectors whose individual values require decoding. Both retain the same file, header, and record context, selecting version-specific registries and validation overlays from `fileFormat` [27]. Our assessment distinguishes a documented representation, an example instantiating it, and an executed query agreeing with an expected answer. Only the last establishes correctness within the tested scope; artifact inspection provides a separate basis for RQ4.

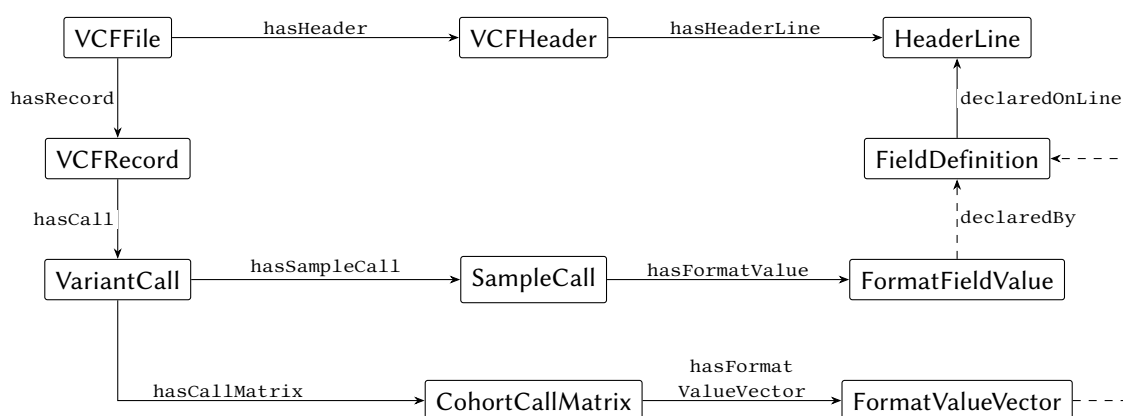


Figure 1: Selected VCF Core relationships. Expanded calls expose individual sample values; condensed calls hold sample-ordered vectors. Both retain file, record, and declaration context. Dashed links identify the declaration supplying a value’s type and cardinality.

²<https://ecrum19.github.io/vcf-core-vocabulary/>

2.2. RQ1: specification-derived assessment and curated inventory

We assess correspondence between VCF specification and VCF Core in both directions: from specification requirements to their *vcfcv* representation, and from the vocabulary to a curated inventory of source constructs. These approaches have precedent in previous ontology assessment studies [33, 34]. The first assessment starts from pinned, digest-verified VCF 4.1–4.5 specifications. A *requirement* expresses a specification rule extracted as a question with a checkable answer, following competency-question and requirements-specification established practices [35, 36, 37]. For example, R16 asks: “Can Number=A values be associated with the correct ALT allele?” Requirements retain links to their source passages. We deemed binary encoding, compression, indexing, producer estimation algorithms, and general invalid-input rejection are outside the scope of this assessment.

Cases and expected answers. A *case* pairs one requirement and VCF version with a *fixture*: a small VCF file designed to expose the rule. The fixture is materialized into a *witness* graph and queried using SPARQL [38]. Each case has an *expected answer* authored from the specification and fixture before query execution, adapting SAMOD and executable ontology-testing approaches [39, 40, 41]. Queries were drafted with AI assistance and reviewed by a human. Expected answers primarily draw on specification examples. Answers are compared as multisets of string-valued rows, so missing, extra, or duplicated rows fail, and ordering is tested by projecting ordinals. Thus, values such as 30 and 30.0 remain distinct. The fixture’s declared VCF version must match its case.

Retrieval, controls, and scoring. Each case runs in both sample profiles on two axes. *Preservation* permits decoding values from literals, whereas *structure* requires graph relationships and prohibits the string-decomposition functions REPLACE, SUBSTR, STRBEFORE, and STRAFTER. An empty-graph control rejects queries that return the expected answer without data. A predicate-deletion control requires the answer to change after deleting at least one referenced property; it establishes dependence on some vocabulary data, rather than on every property mentioned. Requirements are scored per applicable version, profile, and axis as demonstrated when all cases pass, partial when some pass, not demonstrated when all fail, and unassessed when no case exists. Only demonstrated requirements enter the numerator; all applicable requirements, including unassessed ones, remain in the denominator with equal weight.

The VCF Core vocabulary repository’s *Python materializer*³ builds witnesses from fixtures. The materializer, vocabulary, and queries share authors, so a common misunderstanding remains possible despite separately authored answers and the inclusion of controls. RQ2 therefore applies the same assessment to a separate set of witnesses generated an independent implementation to mitigate this potential flaw. Separately, reserved INFO/FORMAT Number/Type declarations are extracted from the specification texts and compared with the corresponding version registries. This tests explicit declarations, independently of prose interpretation.

Curated inventory. The second assessment links 104 VCF 4.5 logical constructs to vocabulary terms and supporting evidence. Reviewers classify whether information is preserved, structurally retrievable, and enforced by a validation rule. These evaluation approaches have also been used to assess semantic ontologies in past studies [33, 34]. Representation is full when both preservation and structure are supported, partial when only preservation is supported, and absent otherwise. Enforcement is recorded separately and does not contribute to the representation score. These curated judgments complement the executed VCF specification requirement tests.

2.3. RQ2: comparison with an independent RDF converter

We re-execute all RQ1 cases against witness graphs generated by VCF-RDFizer 3.0.3 [42], keeping fixtures, queries, and expected answers unchanged. This converter executes RML mappings with

³https://github.com/ecrum19/vcf-core-vocabulary/blob/main/scripts/vcf_examples.py

RMLStreamer, whereas the *vcfcv* materializer constructs triples via a Python script. The programs share an author, so the comparison tests separate implementations without constituting replication by an unrelated group. Every fixture is converted in both profiles, and both producers' graphs are generated during the run. For each case, axis, and profile, we record whether both pass, either producer alone passes, or both fail. We also compare the two profiles case by case. Because producers mint different resource bases, fragment-bearing IRIs are compared by fragment alone; a wrong namespace with the correct fragment could therefore escape detection.

Semantic round-trip. A complementary check reconstructs the seven fixed columns, sample genotypes, header lines, INFO entries, and non-GT FORMAT values from structured RDF properties and compares them with the source. Use of the preserved-source properties `infoRaw`, `sampleDataRaw`, and `genotypeString` are forbidden. Deliberate alterations or deletions of genotype, field, and header data verify that mismatches are detected. A leading phase indicator is normalized because VCF 4.4-5 permits equivalent forms such as `|0|1` and `0|1`; allele order, missing calls, ploidy, and intervening phase indicators must still match exactly. The check uses only expanded graphs because condensed decoding requires an accessor the converter does not yet emit. RQ1 and RQ2 use exact comparison, without sampling or tolerance thresholds.

2.4. RQ3: integration under local alleles

RQ3 tests whether the representation supports a reproducible join to external statements while retaining source context. A synthetic VCF contains four sites represented both locally and globally, with one sample, and is converted in the expanded profile. Its graph is combined in memory with a separately authored local snapshot of five invented, flagged alterations. The two inputs share no identifiers. Listing 1 joins them on contig, position, reference allele, and alternate allele, then retrieves supporting depth through `forAllele`. This assumes the same reference assembly, unchecked against the VCF reference declaration, and identical as-written alleles without normalization or VRS identification. The sample identifies a VCF column, not a patient.

Listing 1: The integration *query*, abridged; `forAllele` resolves the Number=LR frame of reference.

```
# 1. the flagged alteration, from the external annotation snapshot
?annotation a ex:FlaggedAlteration ; ex:contig ?contig ; ex:position ?position ;
           ex:referenceAllele ?ref ; ex:alternateAllele ?alt .
# 2. the same alteration as written in the VCF file
?record vcfc:chrom ?contig ; vcfc:pos ?position ; vcfc:ref ?ref ;
       vcfc:hasCall ?call ; vcfc:hasAltAllele ?altAllele .
?altAllele vcfc:alleleValue ?alt .
# 3. that sample's LAD value, and the declaration giving its cardinality
?call vcfc:hasSampleCall/vcfc:hasFormatValue ?field .
?field vcfc:declaredBy [ vcfc:fieldId "LAD" ; vcfc:fieldNumber ?number ] ;
       vcfc:hasValueItem ?item .
# 4. the depth for THIS allele, via the sample's own local allele set
?item vcfc:forAllele ?altAllele ; vcfc:itemValue ?depth .
```

The query selects flagged alterations with `LAD >= 20`, a threshold chosen to distinguish correct local-allele interpretation from positional misreading. Please note, this example was chosen semi-arbitrarily and does not exhibit clinical significance. Expected rows are authored before execution and compared exactly across all eight projected fields, rather than by row count. Each of the three excluded alterations is checked individually, and every returned row must cite the file actually converted, verified against the input path. The example is restricted to the Number=LR field LAD; its annotations have no biological meaning and are held locally for deterministic execution.

2.5. RQ4: comparison with existing models

One reviewer inspected published artifacts for GFVO, HERO-Genomics, GVO, and VCF2RDF against nine retrieval criteria (Table 4) [17, 24, 20, 18]. Verdicts are **E**, explicitly represented; **C**, representable with additional conventions; and **S**, outside stated scope. The criteria concern VCF interpretation context, and **S** denotes a difference of purpose rather than a defect. Artifact digests identify the inspected versions; VCF2RDF's terms were inspected, while converter behaviour was taken from its paper rather than execution. GA4GH VRS is not scored because it addresses computable variation identity rather than source-file representation [15]. This assessment measures documented support, not the capabilities of extensions or applications built around these models.

Evaluated artifacts and reproduction. All results use released VCF Core 2.1.2 [27] and VCF-RDFizer 3.0.3, with the latter's container pinned by digest. The [experimental repository](#)⁴ provides requirements, fixtures, queries, expected answers, per-RQ procedures, run scripts, and pinned artifacts in PINS.md. All references to "local" files or results can be found in the experimental repository listed above. Runs record specification digests, commands, graphs, results, and environment information, including the dependency rdflib 7.6.0 for query execution.

3. Results

3.1. RQ1: coverage of VCF constructs and meaning

Declarations and assessment corpus. All 333 explicit reserved Number/Type declarations extracted from VCF 4.1–4.5 match the version registries, with no missing or differing row. VCF 4.5's registry contains 122 reserved definitions: 51 INFO and 71 FORMAT, including three parameterized base-modification families. These are different populations: the comparison includes only explicit, comparable declarations, while older specifications also define reserved keys in prose without complete Number/Type information. Agreement is therefore weighted towards VCF 4.3–4.5 and does not settle declaration–prose inconsistencies, such as CNL/CNP declaring Number=G while describing copy-number ordering.

The source audit yielded **94 requirements, 38 fixtures, 210 cases, and 79 distinct queries**, linked to specification passages and expected answers. Nine fixtures reproduce published specification examples exactly; the others are minimal files instantiating stated rules. Availability of defensible examples influenced test selection, leaving the demonstrated subset uneven across the specification. Interpretation statements also vary in specificity: 56 are requirement-specific, 29 generic, and nine explicitly unassessed. During our review we identified inconsistent specification examples, including three issues (such as [hts-specs#869](#)⁵) reported to the maintainers and one previously reported issue, documented in the experimental repository. Where an interpretation could be established, affected requirements were scored; unresolved cases remained unassessed and counted against coverage.

Executed coverage. The 210 cases exercise **53 of 94 requirements** in at least one applicable version; 41 have no registered test. All 210 preservation tests passed in each profile, as did all 210 expanded-structure tests. Condensed structure passed 165 and failed 45 tests, all concerning sample-level FORMAT access held in vectors rather than individual resources. Every passing case rejected the empty-graph control and changed its answer under at least one referenced-predicate deletion. Table 2 reports requirements rather than cases, separately by applicable VCF version.

For VCF 4.5, 51 of 91 requirements (56.0%) are demonstrated in the expanded profile. Of the 40 unassessed requirements, 37 await fixtures and queries, one requires further interpretation, and two require coverage of all reserved INFO and FORMAT keys. Thus, most missing coverage reflects absent

⁴https://github.com/ecrum19/SWAT4HCLS_2027

⁵<https://github.com/samtools/hts-specs/issues/869>

Table 2

Demonstrated requirements by VCF version, sample profile, and retrieval axis. Each Demonstrated column assesses the same N applicable requirements: demonstrated + ND + U = N , with percentages calculated over N . U denotes untested requirements; ND denotes tested failures and is nonzero only for condensed structure ([†]). No requirement was partially demonstrated.

Demonstrated							
VCF	N	<i>preservation</i>		<i>structure</i>		ND [†]	U
		<i>expanded</i>	<i>condensed</i>	<i>expanded</i>	<i>condensed</i>		
4.1	69	32 (46.4%)	32 (46.4%)	32 (46.4%)	26 (37.7%)	6	37
4.2	72	34 (47.2%)	34 (47.2%)	34 (47.2%)	28 (38.9%)	6	38
4.3	74	35 (47.3%)	35 (47.3%)	35 (47.3%)	29 (39.2%)	6	39
4.4	83	38 (45.8%)	38 (45.8%)	38 (45.8%)	31 (37.3%)	7	45
4.5	91	51 (56.0%)	51 (56.0%)	51 (56.0%)	37 (40.7%)	14	40

tests; the tested structural failures concern the condensed sample representation. The corresponding expanded tests pass, demonstrating that those values can be expressed structurally within VCF Core.

Curated inventory. All **104** inventoried VCF 4.5 logical constructs were classified as preserved and structured; **87** additionally had a rule enforcing a stated requirement [27]. Enforcement was less complete for headers, lexical rules, genotypes, and structural variation. Evidence included queries, regression probes, shapes or decoded checks, and materialized fixtures. These are curated representation judgments supported by different evidence types, rather than 104 independently executed retrieval tests.

3.2. RQ2: behaviour across producers and profiles

Re-executing the 210 cases on two axes and two profiles gives **840 checks: 693 pass for both producers, 45 fail for both, and 102 differ** across 15 requirements. Every disagreement is a materializer pass and converter failure; none shows the inverse. The 45 shared failures are condensed-structure queries requiring per-sample resources. Of the 53 exercised requirements, 38 agree on every check and 32 pass on every check. Inspection of each disagreement identifies the causes in Table 3. No observed divergence requires a representational mechanism absent from the pinned vocabulary; every missing property is already declared.

Table 3

Causes of disagreement across 15 requirements.

Cause	Reqs.	Finding
Unemitted properties	12	Twenty declared properties absent, including accessors for gVCF blocks, breakend mates, and allele indices.
Incomplete decomposition	1	Tandem-repeat summary emitted without per-allele components.
Permitted datatype difference	1	QUAL uses <code>vcfc:VCFFloat</code> or <code>xsd:decimal</code> ; both satisfy the shape but differ under <code>DATATYPE()</code> .
Encoding convention	1	Different handling of an empty local-allele list (R27).

An earlier comparison also exposed a silent local-allele error missed by the converter’s existing tests. It associated `Number=LA` and `Number=LR` values with record alleles positionally instead of resolving the sample’s local subset. Under `LAA=2, 4`, depths 20, 30, 10 belong to global alleles 0, 2, and 4, but were attached to 0, 1, and 2. That defect and eight further divergences were repaired [42]. In the evaluated release, all local-allele requirements pass for both producers on expanded-profile checks; the condensed-profile R27 divergence remains.

Semantic round-trip and profile comparison. Across 38 fixtures, reconstruction from structured properties recovers the seven fixed columns of all **109 records**, all **151 sample genotypes**, all **431 header lines**, and **400 of 401** INFO entries and non-GT FORMAT subfields. The exception is an empty LAA list: the materializer writes an empty value, while the converter emits the field without a value triple. This loses the distinction between a present, empty list, meaning reference-only local alleles, and a field without a value. The round-trip therefore reaches R27 through a complementary check. The associated LAD item retains its `forAllele` link, so the error does not remove that depth association.

The sample profiles agree on **352 of 420** case-and-axis comparisons. All 68 disagreements concern FORMAT values or genotypes within samples, spanning 15 requirements. Questions about files, headers, records, and alleles agree throughout. The profiles therefore agree above the sample level, while the condensed vectors require decoding that direct structural queries do not perform.

3.3. RQ3: integration with retained source context

The query returns exactly the two expected alterations: `chr1:1 G>C` at depth 30 and `chr1:2 A>T` at depth 25. Each is traced to its file, record, sample, and the FORMAT declaration defining LAD's cardinality, and every row identifies the file actually converted. All three expected exclusions are confirmed individually: `chr1:1 G>A`, `chr1:2 A>C`, and `chr1:3 C>G` are outside the respective local-allele subsets; the last would also fail the depth threshold under the positional reading.

A positional interpretation of `Number=LR` also returns two rows with provenance, but associates the depths with `chr1:1 G>A` and `chr1:2 A>C` instead. Thus, plausible values, the expected row count, and provenance can coexist with incorrect allele associations. Two exclusions change only when the sample-specific LAA mapping is resolved. This counterexample was defined from the fixture and specification rule before query execution; it illustrates the consequence of the rule, rather than measuring an error in a particular converter. The result demonstrates the utility of local-allele-aware representation for this synthetic example, with no claim of clinical relevance or correctness across all VCF data.

3.4. RQ4: relation to existing VCF semantic models

Table 4 summarizes the inspected artifacts. HERO-Genomics and GVO retain INFO as one literal through `hero:vcfInfo` and `gvo:info`, requiring consumers to parse the text and supply interpretation rules [24, 20]. GFVO directly models field meanings such as `AlleleCount` and `MappingQuality`, but leaves the declaration mechanism outside its scope [17]. VCF2RDF publishes declaration terms, including `Number`, `Type`, and `Description`, but no allele-index term, so allele association requires additional conventions [18]. GVO's predominantly S verdicts reflect its focus on classifying biological variation rather than representing source-file relationships.

Table 4

Support in inspected artifacts: E, explicitly represented; C, requiring additional conventions; S, outside stated scope. These are inspection judgments about VCF interpretation context.

Can a consumer...	VCF Core	GFVO	HERO	GVO	VCF2RDF
F1 trace a record to its file	E	E	E	S	C
F2 read declaration ID, Number, and Type	E	S	S	S	E
F3 read the declared VCF version	E	S	S	S	C
F4 address ALT allele <i>n</i> by index	E	C	C	S	C
F5 separate ploidy, allele calls, and phasing	E	C	C	S	C
F6 get a (sample, field) value unsplit	E	C	C	S	C
F7 bind a <code>Number=A/R/G</code> item to its allele	E	C	C	S	C
F8 distinguish explicit . from unstated	E	S	C	C	C
F9 identify the sample encoding	E	S	S	S	S

VCF Core’s distinguishing role is the explicit representation of source interpretation context: declarations, version rules, allele indices, and sample scope. These findings concern the published artifacts and do not show that other models cannot support the same information through extensions. Separately, VCF Core provides **123 curated alignments**⁶, recorded in SSSOM with SKOS predicates: 57 exact, 40 close, 16 related, and ten narrow matches [43, 44]. These graded term mappings support connections to other models without establishing identity between instances.

4. Discussion

4.1. Assessing VCF Core’s coverage of VCF meaning

This study assesses whether VCF Core makes the information needed to interpret VCF observations available for reuse in RDF. Preserving a field as text does not establish that its declaration, allele ordering, and sample context are accessible to a graph query. RQ1 therefore tests preservation and structured retrieval, while RQ2 examines whether the same answers can be obtained from a separately implemented converter among other assessments designed to test the robustness of *vcfcv*’s stated goal of faithfully representing the VCF specification. The results support broad representation with incomplete demonstrated coverage: the available expanded-profile tests pass, but many requirements remain unassessed (Table 2). The lower condensed-structure scores reflect a different access mechanism, since individual sample values require decoding. For users, this distinction makes coverage meaningful in relation to both the information needed and the way an application must retrieve it.

4.2. Extending demonstrated coverage

The clearest opportunity to improve coverage is to extend testing of structural-variant fields, allele-dependent values, and version-specific rules. New fixtures should isolate each rule, establish expected answers from the specification, and exercise all applicable versions [39, 40]. Validation also needs to extend to constructs without enforcement evidence. Because clear specification examples guided the current tests, the demonstrated subset should not be treated as representative of all VCF complexity.

Deriving executable questions also exposed inconsistencies in specification examples and declarations. This makes vocabulary assessment useful for specification maintenance: an executable example gives maintainers a concrete interpretation to review. Similar benefits have been reported for protocol specifications and genomics-format conformance [45, 46], while unstated VCF conventions have also complicated reuse [47]. Clarifying these rules, refining generic requirement descriptions, and testing files from different production pipelines would strengthen the evidence beyond the present fixtures.

4.3. A reusable assessment for conversion engines

RQ2 shows how the same assessment can guide converter development. The corrected local-allele error produced plausible values attached to the wrong alleles, while the remaining empty-list mismatch was detected through complementary query and round-trip checks. These findings demonstrate why validation of graph structure should be accompanied by checks of recovered values and their associations.

A shared vocabulary makes those checks reusable across conversion strategies. Implementers can develop streaming, mapping-based, or materializing converters against the same representation, while users can evaluate the fields and sample profile their application requires before comparing runtime or graph size. The current comparison supports this approach across two separate implementations, although their shared authorship leaves open the possibility of a common misunderstanding. Re-execution by unrelated groups is therefore a necessary next step.

⁶<https://github.com/ecrum19/vcf-core-vocabulary/tree/main/mappings>

4.4. Integrating observations with external knowledge

We chose local alleles for RQ3 because they make the importance of interpretation context particularly clear. A record can list alternate alleles observed across a cohort, while a sample's LAA identifies a subset. Depths in LAD, declared with `Number=LR`, refer to the reference allele and that local subset. Retrieving depth for an externally annotated alteration therefore depends on the declaration, the sample mapping, and the record allele together. The specification recommends abstracting this encoding within libraries [6]; VCF Core exposes the resolved association through `forAllele`.

The preliminary findings are encouraging because this relationship changes which alterations satisfy the query, even when both interpretations yield plausible rows with provenance. Its practical value is that the interpretation can be implemented and checked during conversion, then reused by downstream queries without decoding local alleles again. The synthetic example assumes a shared assembly and identical written alleles; it demonstrates this capability while leaving realistic biomedical integration and reference-aware normalization to further evaluation.

4.5. VCF Core among existing models

RQ4 identifies VCF Core's distinct role in making source interpretation context directly addressable. The inspected models serve complementary purposes, while some VCF relationships require parsing or additional conventions (Table 4). The criteria reflect the problems VCF Core was designed to address, so the comparison should be read within that scope.

Observations from two laboratories could retain their own quality values, sample context, and provenance while linking, after normalization, to a common variation description such as GA4GH VRS [15]. A clinical model such as HERO-Genomics could connect sample columns to specimens and patients [24]; Med2RDF and *gvar* provide further candidates for alignment [48, 49]. The published mappings provide graded relationships between terms, rather than identity between instances. Review by the respective model communities would strengthen these connections and the inspection-based comparison [3].

4.6. Future work and independent evaluation

Future studies should test this foundation in workflows linking pharmacogenomic interpretations to supporting observations [50, 51], rare-disease candidates to phenotype knowledge [52, 2], or tumour observations to therapeutic evidence [53]. These proposed scenarios require biomedical models, reviewed instance mappings, and explicit sample-specimen-patient relationships. Their value should be assessed through answer correctness and traceability.

Realistic cohort studies should also measure conversion, storage, loading, and query costs alongside correctness. Current round-trip and integration results apply only to expanded data; SPARQL extensions such as SciSPARQL [54] could support investigation of access to condensed vectors. We invite independent review of the vocabulary, converters, requirement interpretations, and expected answers. New fixtures, counterexamples, and disputed interpretations would directly improve this resource and its basis in the VCF specification.

5. Conclusion

VCF Core provides a shared semantic foundation for representing VCF observations together with the context needed to interpret them. The assessment demonstrates structured retrieval for the tested requirements, reuse across two separately implemented producers, and an allele-dependent integration task with checked answers and source provenance. The remaining gaps concern untested requirements, incomplete converter support, and evaluation beyond synthetic examples. By making declarations, allele relationships, and sample context explicit, the vocabulary allows conversion engines and downstream queries to reuse the same interpretation rules while connecting source observations to biomedical

models. We invite independent review and testing to extend the evidence for this resource and clarify where further development is needed.

Acknowledgments

Project funding provided from the Research Foundation – Flanders (FWO) (SB Fellowship 1S27825N). Ruben Taelman is a postdoctoral fellow of the Research Foundation – Flanders (FWO) (1202124N).

Declaration on Generative AI

GPT-6 Astra (OpenAI) and Claude Opus 5 (Anthropic) assisted with the development of the VCF Core Vocabulary, the scripts used throughout this study, the initial writing of SPARQL queries used for testing, and the refinement of manuscript figures and text. The authors reviewed all AI outputs, all code generated by AI models, and all AI manuscript revisions to the manuscript. The authors take full responsibility for all content, tools, repositories, and paper contents.

References

- [1] C. Gaudet-Blavignac, J. L. Raisaro, V. Touré, S. Österle, K. Cramer, C. Lovis, A national, semantic-driven, three-pillar strategy to enable health data secondary usage interoperability for research within the Swiss Personalized Health Network: Methodological study, *JMIR Medical Informatics* 9 (2021) e27591. URL: <https://medinform.jmir.org/2021/6/e27591/>. doi:10.2196/27591.
- [2] T. E. Putman, K. Schaper, N. Matentzoglou, V. P. Rubinetti, F. S. Alquaddoomi, C. Cox, J. H. Caufield, G. Elsarboukh, S. Gehrke, H. Hegde, J. T. Reese, I. Braun, R. M. Bruskewich, L. Cappelletti, S. Carbon, A. R. Caron, L. E. Chan, C. G. Chute, K. G. Cortes, V. De Souza, T. Fontana, N. L. Harris, E. L. Hartley, E. Hurwitz, J. O. B. Jacobsen, M. Krishnamurthy, B. J. Laraway, J. A. McLaughlin, J. A. McMurry, S. A. T. Moxon, K. R. Mullen, S. T. O’Neil, K. A. Shefchek, R. Stefancsik, S. Toro, N. A. Vasilevsky, R. L. Walls, P. L. Whetzel, D. Osumi-Sutherland, D. Smedley, P. N. Robinson, C. J. Mungall, M. A. Haendel, M. C. Munoz-Torres, The Monarch Initiative in 2024: an analytic platform integrating phenotypes, genes and diseases across species, *Nucleic Acids Research* 52 (2024) D938–D949. URL: <https://academic.oup.com/nar/article/52/D1/D938/7449493>. doi:10.1093/nar/gkad1082.
- [3] B. Smith, M. Ashburner, C. Rosse, J. Bard, W. Bug, W. Ceusters, L. J. Goldberg, K. Eilbeck, A. Ireland, C. J. Mungall, The OBI Consortium, N. Leontis, P. Rocca-Serra, A. Ruttenberg, S.-A. Sansone, R. H. Scheuermann, N. Shah, P. L. Whetzel, S. Lewis, The OBO Foundry: coordinated evolution of ontologies to support biomedical data integration, *Nature Biotechnology* 25 (2007) 1251–1255. URL: <https://www.nature.com/articles/nbt1346>. doi:10.1038/nbt1346.
- [4] F. Belleau, M.-A. Nolin, N. Tourigny, P. Rigault, J. Morissette, Bio2RDF: Towards a mashup to build bioinformatics knowledge systems, *Journal of Biomedical Informatics* 41 (2008) 706–716. URL: <https://www.sciencedirect.com/science/article/pii/S1532046408000415>. doi:10.1016/j.jbi.2008.03.004.
- [5] P. Danecek, A. Auton, G. Abecasis, C. A. Albers, E. Banks, M. A. DePristo, R. E. Handsaker, G. Lunter, G. T. Marth, S. T. Sherry, G. McVean, R. Durbin, 1000 Genomes Project Analysis Group, The variant call format and VCFtools, *Bioinformatics* 27 (2011) 2156–2158. URL: <https://academic.oup.com/bioinformatics/article/27/15/2156/402296>. doi:10.1093/bioinformatics/btr330.
- [6] VCF, The Variant Call Format Specification: VCFv4.5 and BCFv2.2, 2026. URL: <https://samtools.github.io/hts-specs/VCFv4.5.pdf>, 25 February 2026 printing, version e821e4f; accessed 6 September 2026.
- [7] Global Alliance for Genomics and Health, HTS Specifications: SAM/BAM and Related High-Throughput Sequencing File Formats, <https://github.com/samtools/hts-specs>, n.d. Repository containing the Variant Call Format specifications, including VCF versions 4.1–4.5.

- [8] SAM/BAM Format Specification Working Group, HTS format specifications: VCF specifications and earlier versions, Official specification index, 2026. URL: <https://samtools.github.io/hts-specs/>, accessed 9 September 2026.
- [9] S. Decker, S. Melnik, F. van Harmelen, D. Fensel, M. Klein, J. Broekstra, M. Erdmann, I. Horrocks, The semantic web: the roles of XML and RDF, *IEEE Internet Computing* 4 (2000) 63–73. URL: <https://ieeexplore.ieee.org/document/877487/>. doi:10.1109/4236.877487.
- [10] C. Bizer, T. Heath, T. Berners-Lee, Linked Data—the story so far, *International Journal on Semantic Web and Information Systems* 5 (2009) 1–22. URL: <https://doi.org/10.4018/jswis.2009081901>. doi:10.4018/jswis.2009081901.
- [11] I. M. Lubin, N. Aziz, L. J. Babb, D. Ballinger, H. Bisht, D. M. Church, S. Cordes, K. Eilbeck, F. Hyland, L. Kalman, M. Landrum, E. R. Lockhart, D. Maglott, G. Marth, J. D. Pfeifer, H. L. Rehm, S. Roy, Z. Tezak, R. Truty, M. Ullman-Cullere, K. V. Voelkerding, E. A. Worthey, A. W. Zaranek, J. M. Zook, Principles and recommendations for standardizing the use of the next-generation sequencing variant file in clinical settings, *The Journal of Molecular Diagnostics* 19 (2017) 417–426. URL: <https://doi.org/10.1016/j.jmoldx.2016.12.001>. doi:10.1016/j.jmoldx.2016.12.001.
- [12] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, G. Appleton, M. Axton, A. Baak, N. Blomberg, J.-W. Boiten, L. B. da Silva Santos, P. E. Bourne, J. Bouwman, A. J. Brookes, T. Clark, M. Crosas, I. Dillo, O. Dumon, S. Edmunds, C. T. Evelo, R. Finkers, A. Gonzalez-Beltran, A. J. G. Gray, P. Groth, C. Goble, J. S. Grethe, J. Heringa, P. A. C. 't Hoen, R. Hooft, T. Kuhn, R. Kok, J. Kok, S. J. Lusher, M. E. Martone, A. Mons, A. L. Packer, B. Persson, P. Rocca-Serra, M. Roos, R. van Schaik, S.-A. Sansone, E. Schultes, T. Sengstag, T. Slater, G. Strawn, M. A. Swertz, M. Thompson, J. van der Lei, E. van Mulligen, J. Velterop, A. Waagmeester, P. Wittenburg, K. Wolstencroft, J. Zhao, B. Mons, The FAIR guiding principles for scientific data management and stewardship, *Scientific Data* 3 (2016) 160018. URL: <https://www.nature.com/articles/sdata201618>. doi:10.1038/sdata.2016.18.
- [13] C. Bahim, C. Casorrán-Amilburu, M. Dekkers, E. Herczog, N. Loozen, K. Repanas, K. Russell, S. Stall, The FAIR Data Maturity Model: An approach to harmonise FAIR assessments, *Data Science Journal* 19 (2020) 41. URL: <https://datascience.codata.org/articles/dsj-2020-041>. doi:10.5334/dsj-2020-041.
- [14] E. Garrison, Z. N. Kronenberg, E. T. Dawson, B. S. Pedersen, P. Prins, A spectrum of free software tools for processing the VCF variant call format: vcflib, bio-vcf, cyvcf2, hts-nim and slivar, *PLOS Computational Biology* 18 (2022) e1009123. URL: <https://doi.org/10.1371/journal.pcbi.1009123>. doi:10.1371/journal.pcbi.1009123.
- [15] A. H. Wagner, L. Babb, G. Alterovitz, M. Baudis, M. Brush, D. L. Cameron, M. Cline, M. Griffith, O. L. Griffith, S. E. Hunt, D. Kreda, J. M. Lee, S. Li, J. Lopez, E. Moyer, T. Nelson, R. Y. Patel, K. Riehle, P. N. Robinson, S. Rynearson, H. Schuilenburg, K. Tsukanov, B. Walsh, M. Konopko, H. L. Rehm, A. D. Yates, R. R. Freimuth, R. K. Hart, The GA4GH Variation Representation Specification: A computational framework for variation representation and federated identification, *Cell Genomics* 1 (2021) 100027. doi:10.1016/j.xgen.2021.100027.
- [16] T. Katayama, S. Kawashima, G. Micklem, S. Kawano, J.-D. Kim, S. Kocbek, S. Okamoto, Y. Wang, H. Wu, A. Yamaguchi, et al., Biohackathon series in 2013 and 2014: improvements of semantic interoperability in life science data and services, *F1000Research* 8 (2019) 1677. URL: <https://doi.org/10.12688/f1000research.18238.1>. doi:10.12688/f1000research.18238.1.
- [17] J. Baran, B. S. B. Durgahee, K. Eilbeck, E. Antezana, R. Hoehndorf, M. Dumontier, GFVO: the Genomic Feature and Variation Ontology, *PeerJ* 3 (2015) e933. URL: <https://peerj.com/articles/933/>. doi:10.7717/peerj.933.
- [18] E. D. S. Penha, E. Iriabho, A. Dussaq, D. M. de Oliveira, J. S. Almeida, Isomorphic semantic mapping of variant call format (VCF2RDF), *Bioinformatics* 33 (2017) 547–548. URL: <https://academic.oup.com/bioinformatics/article/33/4/547/2593587>. doi:10.1093/bioinformatics/btw652.
- [19] N. Mitsuhashi, L. Toyo-oka, T. Katayama, M. Kawashima, S. Kawashima, K. Miyazaki, T. Takagi, TogoVar: A comprehensive japanese genetic variation database, *Human Genome Variation* 9 (2022) 44. URL: <https://doi.org/10.1038/s41439-022-00222-9>. doi:10.1038/s41439-022-00222-9.

- [20] S. Kawashima, T. Fujisawa, T. Katayama, Genome variation ontology for annotation of complex structural variations, in: *Proceedings of the 14th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2023)*, volume 3415 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2023, pp. 126–127. URL: <https://ceur-ws.org/Vol-3415/paper-22.pdf>.
- [21] S. Prasanna, A. Kumar, D. Rao, E. J. Simoes, P. Rao, A scalable tool for analyzing genomic variants of humans using knowledge graphs and graph machine learning, *Frontiers in Big Data* 7 (2025) 1466391. URL: <https://doi.org/10.3389/fdata.2024.1466391>. doi:10.3389/fdata.2024.1466391.
- [22] R. Janssen, UMC Utrecht Genetics, SPARQLing genomics, Version 0.99.11. Open-source software, 2020. URL: <https://gitlab.com/roelj/sparqling-genomics>, ontologies, RDF generators and a query interface for genomics; GPL-3.0. Released 19 June 2020.
- [23] A. Bodrug-Schepers, H. Chabane, G. Montoya, P. Serrano-Alvarado, R. Redon, A. Gaignard, Semantic beacons: a framework to support federated querying over genomic variants and public knowledge graphs, in: *Proceedings of the 16th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, volume 4196 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2025, pp. 21–32. URL: https://ceur-ws.org/Vol-4196/paper_16.pdf.
- [24] M. Cazzaro, I. G. Gut, L. Menotti, M. Rueda, G. Silvello, HERO-Genomics: An ontology for integration and access of multicenter genomic data, in: *Proceedings of the 16th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, volume 4196 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2025, pp. 33–42. URL: https://ceur-ws.org/Vol-4196/paper_17.pdf.
- [25] P. Larmande, B. Pittolat, N. Tando, Y. Pomie, B. G. Happi Happi, V. Guignon, M. Ruiz, AgroLD: a knowledge graph for the plant sciences, *BMC Genomic Data* 26 (2025) 73. URL: <https://doi.org/10.1186/s12863-025-01359-6>. doi:10.1186/s12863-025-01359-6.
- [26] E. Crum, R. Taelman, B. Buelens, G. Ertaylan, R. Verborgh, Semantifying genomic variant data: VCF to RDF conversion framework, in: *Proceedings of the 16th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, volume 4196 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2025, pp. 199–200. URL: https://ceur-ws.org/Vol-4196/paper_6.pdf.
- [27] E. Crum, VCF Core Vocabulary, Version 2.1.2. GitHub repository, 2026. URL: <https://github.com/ecrum19/vcf-core-vocabulary/tree/3f06d18>, version 2.1.2 with the SSSOM alignment set; source snapshot of 14 September 2026.
- [28] World Wide Web Consortium, PROV-O: The PROV Ontology, W3C Recommendation, World Wide Web Consortium, 2013. URL: <https://www.w3.org/TR/2013/REC-prov-o-20130430/>.
- [29] K. Eilbeck, S. E. Lewis, C. J. Mungall, M. Yandell, L. Stein, R. Durbin, M. Ashburner, The sequence ontology: a tool for the unification of genome annotations, *Genome Biology* 6 (2005) R44. URL: <https://link.springer.com/article/10.1186/gb-2005-6-5-r44>. doi:10.1186/gb-2005-6-5-r44.
- [30] J. T. Bolleman, C. J. Mungall, F. Strozzi, J. Baran, M. Dumontier, R. J. P. Bonnal, R. Buels, R. Hoehndorf, T. Fujisawa, T. Katayama, P. J. A. Cock, FALDO: a semantic standard for describing the location of nucleotide and protein feature annotation, *Journal of Biomedical Semantics* 7 (2016) 39. URL: <https://link.springer.com/article/10.1186/s13326-016-0067-z>. doi:10.1186/s13326-016-0067-z.
- [31] World Wide Web Consortium, Shapes Constraint Language (SHACL), W3C Recommendation, World Wide Web Consortium, 2017. URL: <https://www.w3.org/TR/2017/REC-shacl-20170720/>.
- [32] E. Crum, Ontology Companion Generator (OCG), Version 1.4.0. Open-source software, 2026. URL: <https://github.com/ecrum19/ocg/tree/8963aa4>, MIT licence; released 1 September 2026. Documentation at <https://ecrum19.github.io/ocg/>.
- [33] A. Gómez-Pérez, Evaluation of ontologies, *International Journal of Intelligent Systems* 16 (2001) 391–409. doi:10.1002/1098-111X(200103)16:3<391::AID-INT1014>3.0.CO;2-2.

- [34] M. Sabou, M. Fernandez, Ontology (network) evaluation, in: M. C. Suárez-Figueroa, A. Gómez-Pérez, E. Motta, A. Gangemi (Eds.), *Ontology Engineering in a Networked World*, Springer, Berlin, Heidelberg, 2012, pp. 193–212. doi:10.1007/978-3-642-24794-1_9.
- [35] M. Grüninger, M. S. Fox, Methodology for the design and evaluation of ontologies, in: *Proceedings of the Workshop on Basic Ontological Issues in Knowledge Sharing, IJCAI-95*, Montreal, Canada, 1995. URL: <https://www.eil.utoronto.ca/wp-content/uploads/enterprise-modelling/papers/gruninger-ijcai95.pdf>.
- [36] M. C. Suárez-Figueroa, A. Gómez-Pérez, B. Villazón-Terrazas, How to write and use the ontology requirements specification document, in: *On the Move to Meaningful Internet Systems: OTM 2009*, volume 5871 of *Lecture Notes in Computer Science*, Springer, Berlin, Heidelberg, 2009, pp. 966–982. doi:10.1007/978-3-642-05151-7_16.
- [37] M. Poveda-Villalón, A. Fernández-Izquierdo, M. Fernández-López, R. García-Castro, LOT: An industrial oriented ontology engineering framework, *Engineering Applications of Artificial Intelligence* 111 (2022) 104755. doi:10.1016/j.engappai.2022.104755.
- [38] World Wide Web Consortium, SPARQL 1.1 Query Language, W3C Recommendation, World Wide Web Consortium, 2013. URL: <https://www.w3.org/TR/2013/REC-sparql11-query-20130321/>.
- [39] S. Peroni, A simplified agile methodology for ontology development, in: *OWL: Experiences and Directions – Reasoner Evaluation*, volume 10161 of *Lecture Notes in Computer Science*, Springer, Cham, 2017, pp. 55–69. doi:10.1007/978-3-319-54627-8_5.
- [40] E. Blomqvist, A. Seil Sepour, V. Presutti, Ontology testing: Methodology and tool, in: *Knowledge Engineering and Knowledge Management*, volume 7603 of *Lecture Notes in Computer Science*, Springer, Berlin, Heidelberg, 2012, pp. 216–226. doi:10.1007/978-3-642-33876-2_20.
- [41] D. Wiśniewski, J. Potoniec, A. Ławrynowicz, C. M. Keet, Analysis of ontology competency questions and their formalizations in SPARQL-OWL, *Journal of Web Semantics* 59 (2019) 100534. doi:10.1016/j.websem.2019.100534.
- [42] VCF-RDFizer maintainers, VCF-RDFizer, Version 3.0.3. Open-source software, 2026. URL: <https://github.com/ecrum19/VCF-RDFizer/tree/be658a2>, tag v3.0.3; converted and validated 14 September 2026.
- [43] N. Matentzoglou, J. P. Balhoff, S. M. Bello, C. Bizon, M. Brush, T. J. Callahan, C. G. Chute, W. D. Duncan, C. T. Evelo, D. Gabriel, J. Graybeal, A. Gray, B. M. Gyori, M. Haendel, H. Harmse, N. L. Harris, I. Harrow, H. B. Hegde, A. L. Hoyt, C. T. Hoyt, D. Jiao, E. Jiménez-Ruiz, S. Jupp, H. Kim, S. Koehler, T. Liener, Q. Long, J. Malone, J. A. McLaughlin, J. A. McMurphy, S. Moxon, M. C. Munoz-Torres, D. Osumi-Sutherland, J. A. Overton, B. Peters, T. Putman, N. Queralt-Rosinach, K. Shefchek, H. Solbrig, A. Thessen, T. Tudorache, N. Vasilevsky, A. H. Wagner, C. J. Mungall, A simple standard for sharing ontological mappings (SSSOM), *Database* 2022 (2022) baac035. URL: <https://academic.oup.com/database/article/doi/10.1093/database/baac035/6591806>. doi:10.1093/database/baac035.
- [44] World Wide Web Consortium, SKOS Simple Knowledge Organization System Reference, W3C Recommendation, World Wide Web Consortium, 2009. URL: <https://www.w3.org/TR/skos-reference/>.
- [45] S. Bishop, M. Fairbairn, M. Norrish, P. Sewell, M. Smith, K. Wansbrough, Rigorous specification and conformance testing techniques for network protocols, as applied to TCP, UDP, and sockets, in: *Proceedings of the 2005 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications (SIGCOMM)*, ACM, 2005, pp. 265–276. URL: <https://doi.org/10.1145/1080091.1080123>. doi:10.1145/1080091.1080123.
- [46] Y. N. Niu, E. G. Roberts, D. Denisko, M. M. Hoffman, Assessing and assuring interoperability of a genomics file format, *Bioinformatics* 38 (2022) 3327–3336. URL: <https://doi.org/10.1093/bioinformatics/btac327>. doi:10.1093/bioinformatics/btac327.

- [47] S. Beier, A. Fiebig, C. Pommier, I. Liyanage, M. Lange, P. J. Kersey, S. Weise, R. Finkers, B. Koylass, T. Cezard, M. Courtot, B. Contreras-Moreira, G. Naamati, S. Dyer, U. Scholz, Recommendations for the formatting of variant call format (VCF) files to make plant genotyping data FAIR, *F1000Research* 11 (2022) 231. URL: <https://doi.org/10.12688/f1000research.109080.2>. doi:10.12688/f1000research.109080.2, version 2; peer review: 2 approved.
- [48] Med2RDF contributors, Med2RDF ontology, GitHub repository, n.d. URL: <https://github.com/med2rdf/med2rdf-ontology>, accessed 8 September 2026.
- [49] gvar, Genomic variants schema (gvar), GitHub repository, n.d. URL: <https://github.com/swat4hcls-2025-genomic-variation/genomic-variant-schema>, accessed 6 September 2026.
- [50] C. R. Lee, J. A. Luzum, K. Sangkuhl, R. S. Gammal, M. S. Sabatine, C. M. Stein, D. F. Kisor, N. A. Limdi, Y. M. Lee, S. A. Scott, J.-S. Hulot, D. M. Roden, A. Gaedigk, K. E. Caudle, T. E. Klein, J. A. Johnson, A. R. Shuldiner, Clinical pharmacogenetics implementation consortium guideline for CYP2C19 genotype and clopidogrel therapy: 2022 update, *Clinical Pharmacology & Therapeutics* 112 (2022) 959–967. doi:10.1002/cpt.2526.
- [51] PharmCAT contributors, PharmCAT methods, Official documentation, 2026. URL: <https://pharmcat.org/methods/>, accessed 9 September 2026.
- [52] Human Phenotype Ontology Consortium, The human phenotype ontology, Project website and documentation, 2026. URL: <https://hpo.jax.org/>, accessed 9 September 2026.
- [53] M. Griffith, N. C. Spies, K. Krysiak, J. F. McMichael, A. C. Coffman, A. M. Danos, B. J. Ainscough, C. A. Ramirez, D. T. Rieke, L. Kujan, E. K. Barnell, A. H. Wagner, Z. L. Skidmore, A. Wollam, C. J. Liu, M. R. Jones, R. L. Bilski, R. Lesurf, Y.-Y. Feng, N. M. Shah, M. Bonakdar, L. Trani, M. Matlock, A. Ramu, K. M. Campbell, G. C. Spies, A. P. Graubert, K. Gangavarapu, J. M. Eldred, D. E. Larson, J. R. Walker, B. M. Good, C. Wu, A. I. Su, R. Dienstmann, A. A. Margolin, D. Tamborero, N. Lopez-Bigas, S. J. M. Jones, R. Bose, D. H. Spencer, L. D. Wartman, R. K. Wilson, E. R. Mardis, O. L. Griffith, CIViC is a community knowledgebase for expert crowdsourcing the clinical interpretation of variants in cancer, *Nature Genetics* 49 (2017) 170–174. doi:10.1038/ng.3774.
- [54] A. Andrejev, T. Risch, Scientific SPARQL: Semantic web queries over scientific data, in: 2012 IEEE 28th International Conference on Data Engineering Workshops (ICDEW), IEEE, 2012, pp. 5–10. URL: <https://doi.org/10.1109/ICDEW.2012.67>. doi:10.1109/ICDEW.2012.67.